

Base class Pointer :-

Base class Pointer can point to the object of any of its descendant class.

But its converse is not true.

Program: Using base class Pointer to call functions

```
#include <iostream>
using namespace std;
```

```
Class Parent
```

```
{
```

```
public:
```

```
void show()
{
```

```
cout << "This is Parent class show() \n";
```

```
}
```

```
Class child : public Parent
```

```
{
```

```
public:
```

```
void show()
{
```

```
cout << "This is Child class show() \n";
```

```
}
```

```
void main()
```

```
{
```

```
Parent *p, Obj1;
```

```
Child Obj2;
```

```
p = &Obj1
```

p → show(); → To call function through pointer we use → arrow instead of dot.

O/p → In this case output :
This is Parent class show()



$p = \&\text{obj2};$ $\&\text{obj2} \rightarrow$ object of ^{child} class?

$p \rightarrow \text{show}();$

O/p $\rightarrow$

This is Parent class show()?

This is because the function call is decided by the pointer type & which datatype is used to declare the pointer?

This is ~~is~~ called **Compile-time binding** & Early binding?

* Object calls $\rightarrow$ decided by object type

* Pointer calls & without virtual $\rightarrow$ decided by pointer type

So here the concept of virtual function is introduced

A virtual function is a member function of a class that ensures **late binding** & run-time binding so that the function call is resolved at runtime based on the **actual object type**.

Program:

The compiler looks at the base ~~class~~ class declaration to decide whether the call should be resolved:

$\leftarrow$ at compile time

• or at runtime

Program: #include <iostream>
using namespace std;

Class Parent

{

public:

virtual void show()

{

cout << "This is Parent class show () \n";

}

}

Class child : public Parent

{

public:

void show()

{

cout << "This is child class show () \n";

}

}

void main()

{

Parent *p;

Parent obj1;

Child obj2;

p = &obj1;

p -> show();

p = &obj2;

p -> show();

}

DATE



Output :

This is Parent class show()

This is Child class show()